Laboratório Funcional · Aula 12

Gabriel Galhardone Felipe · RA 24021292-2 · Turma Esoft B
Gerado em 16/10/2026, 20:02:06 · tempo registrado no cronômetro: 50:26

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

1. sorteia(): É impura porque utiliza random.randint(1, 6), que gera números aleatórios. Assim, chamadas sucessivas podem retornar resultados diferentes.

2. poe(lista, x): É impura porque utiliza lista.append(x), que modifica a lista original, causando um efeito colateral.

Prática 1: Funções puras e formas funcionais

Previsão: O que as linhas 1, 4 e 7 vão imprimir?

1: A-1 A-2

4: 14

7: [7, 3, 9] [3, 7, 9]

Saídas (original e com compor(g, f))

```
1: A-1 A-2
2: True
3: 8
4: 14
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]
```

1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

função proximo_id("A") devolve valores diferentes porque depende da variável global contador, que é incrementada a cada chamada. Assim, a primeira chamada retorna "A-1" e a segunda retorna "A-2", mesmo recebendo a mesma entrada.

Isso dificulta os testes porque o resultado depende do estado anterior do programa. Para testar a função, é preciso considerar o valor atual de contador e a ordem das chamadas. Isso torna os testes menos previsíveis e mais difíceis de repetir.

Uma função pura, como cubo(x), sempre devolve o mesmo resultado para a mesma entrada, facilitando os testes.

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

compor(f, g)(4):
g
(
4
)
=
12
g(4)=12, depois
f
(
12
)
=
14
f(12)=14.

compor(g, f)(4):
f
(
4
)
=
6
f(4)=6, depois
g
(
6
)
=
18
g(6)=18.

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

sorted(notas) cria uma nova lista ordenada, sem modificar a original. notas.sort() ordena a própria lista, modificando-a.

sorted(notas) segue o estilo funcional porque não altera a lista original, evitando efeitos colaterais.

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

(car '(A B C)) → A

Previsão: (cdr '(A B C))

(cdr '(A B C)) → (B C)

Previsão: (cons '(A B) '(C D))

(cons '(A B) '(C D)) → ((A B) C D)

Saídas (original e sem o apóstrofo)

```
Saída original:
A
(B C)
((A B) C D)

Saída sem o apóstrofo em (car '(A B C)):
Erro: A não é uma função válida.
```

2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

O apóstrofo (') serve para impedir que o Scheme avalie uma lista como uma expressão, tratando-a como dado.

Sem o apóstrofo, o Scheme tentou executar (A B C) como uma chamada de função, considerando A como a função e B e C como parâmetros, causando um erro.

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
(car (cdr (cdr '(A B C))))
O resultado seria → C
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

n = 4: comum em 8 passos, até 3 multiplicações esperando (expressão de até 30 caracteres); em cauda em 6 passos, nada esperando (até 17 caracteres).

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

Com n = 4, as duas formas de calcular o fatorial retornam 24; com n = 6, retornam 720. A recursão comum deixa multiplicações pendentes até as chamadas terminarem, enquanto a recursão em cauda usa um acumulador e não deixa operações pendentes.

No código Scheme, conta-ate pode executar um milhão de chamadas porque a recursão em cauda pode ser

otimizada, dependendo da implementação. Em Python, a mesma função provavelmente excederá o limite de recursão padrão, que é de aproximadamente mil chamadas.

Saída do Scheme

```
#t
#f
2432902008176640000
2432902008176640000
1000000
```

Saída do Python (as últimas linhas bastam)

```
500
RecursionError: maximum recursion depth exceeded
```

3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

Na recursão comum, crescem as operações pendentes e os registros na pilha, porque as chamadas anteriores precisam esperar o resultado das próximas. Na recursão em cauda, não ficam operações pendentes, pois a chamada recursiva é a última operação e pode permitir o reaproveitamento da pilha.

3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

O Scheme otimizou a recursão em cauda, reutilizando a pilha de execução e permitindo um milhão de chamadas. Já o Python tem um limite de recursão e não faz essa otimização automaticamente, por isso parou com RecursionError.

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

```
tamanho: 3  ok
soma-quadrados: 14  ok
pares: (2 4 6)  ok
```

4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
(define (confere nome obtido esperado)
  (display nome) (display ": ") (display obtido)
  (if (equal? obtido esperado)
      (display "  ok")
      (begin (display "  (esperado ") (display esperado) (display ")")))
  (newline))

(define (tamanho lis)
  (cond ((null? lis) 0)
        (else (+ 1 (tamanho (cdr lis))))))

(define (soma-quadrados lis)
  (cond ((null? lis) 0)
        (else (+ (* (car lis) (car lis))
                 (soma-quadrados (cdr lis))))))
```

```
(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis))
         (cons (car lis) (pares (cdr lis))))
        (else (pares (cdr lis)))))

(confere "tamanho" (tamanho '(a b c)) 3)
(confere "soma-quadrados" (soma-quadrados '(1 2 3)) 14)
(confere "pares" (pares '(1 2 3 4 5 6)) '(2 4 6))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

A parte mais difícil foi entender como usar a recursão para percorrer a lista até o final. Em todas as funções, usei null? para verificar se a lista estava vazia. Na função tamanho, usei cdr para avançar na lista e contar os elementos. Na função soma-quadrados, usei car para pegar o primeiro número e calcular seu quadrado, e cdr para continuar com os próximos. Já na função pares, usei car para verificar se o número era par com even? e cdr para percorrer o restante da lista, guardando apenas os números pares.

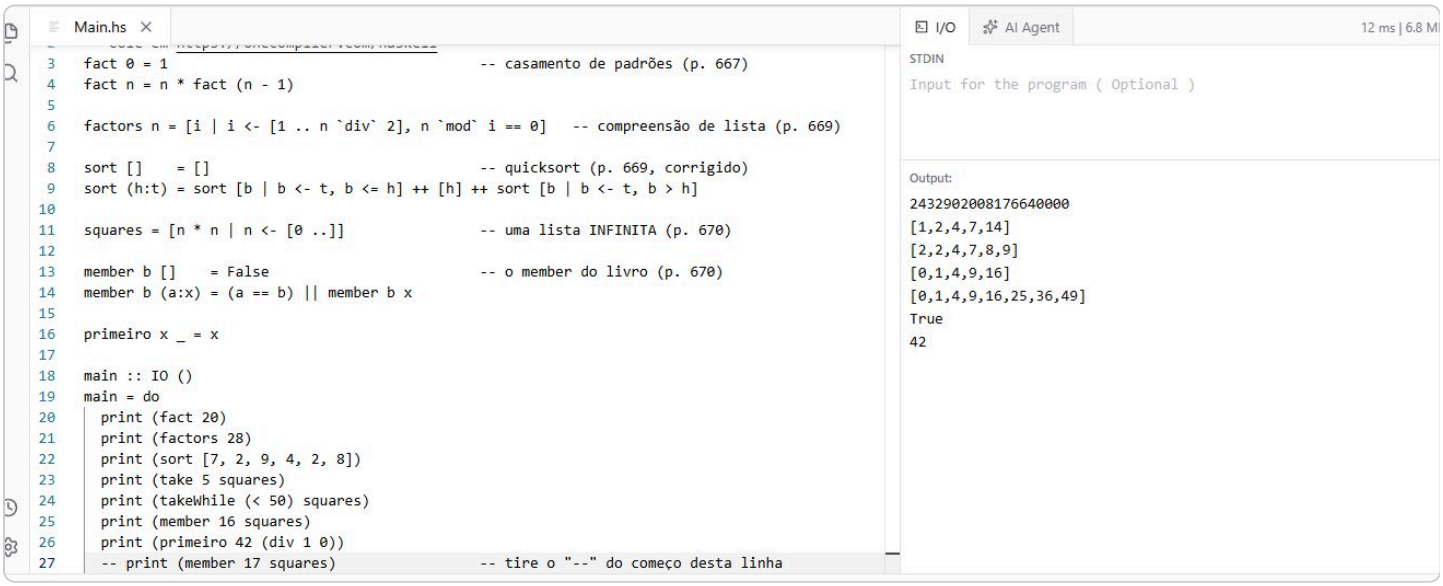
Prática 5: Haskell: padrões, compreensões e preguiça

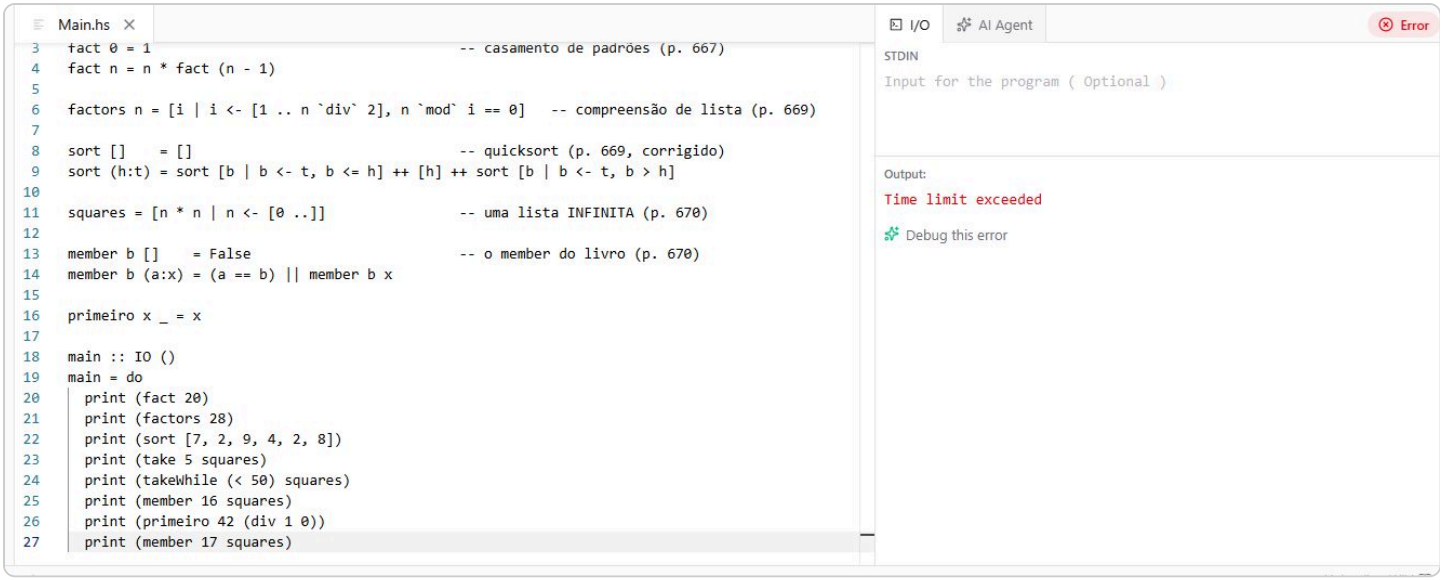
Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

Acredito que o primeiro vai mostrar os cinco primeiros números ao quadrado, começando pelo zero. Já no segundo, acho que vai retornar o número 42, porque a função não precisa usar o segundo valor. Pelo que entendi, como o Haskell só calcula algo quando é necessário, a divisão por zero não chega a ser executada.

Saídas (original e com member 17)

```
print abaixo
```





5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

Pelo que entendi, o Haskell consegue mostrar apenas uma parte da lista infinita porque calcula os valores conforme precisa, sem ter que gerar a lista inteira. Já no caso do primeiro 42 (div 1 0), acredito que não deu erro porque a função só utiliza o número 42 e não precisa calcular a divisão por zero.

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

O member 17 squares nunca termina porque o número 17 não aparece na lista de quadrados perfeitos. Já o member 16 squares termina porque encontra o número 16 na lista. Eu corrigiria a função para ela parar a busca quando o número da lista passar do valor procurado, já que os quadrados estão em ordem crescente. Assim, ela não fica procurando para sempre.

Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

Acredito que mais10(5) vai imprimir 15, porque soma 10 ao número 5. Já o map(vezes(5)) vai multiplicar cada número da lista por 5, então o resultado será uma lista com 5, 10 e 15.

Saída do JavaScript

```
15
[ 5, 10, 15 ]
56
{ nome: 'Ana', nota: 7 } { nome: 'Ana', nota: 8 }
```

Saída do Java

```
14
18
1050.0
[CABO, MONITOR, MOUSE, TECLADO]
```

6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

Pelo que entendi, isso acontece porque a função devolvida guarda uma referência ao ambiente em que foi criada, incluindo o valor 10. Esse comportamento é chamado de fechamento (closure). Assim, mesmo depois que somador(10) termina, a função mais10 continua lembrando desse valor e consegue usá-lo quando é chamada.

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

Os resultados foram diferentes porque a ordem em que as funções são executadas muda. O compose executa primeiro a função g e depois a f, enquanto o andThen executa primeiro a f e depois a g. Acredito que o compose corresponde ao compor(f, g) da Prática 1, pois primeiro aplica g e depois f ao resultado.

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
1 sua saida esperada é 244
```

7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
const alunos = [
  { nome: "Ana", nota: 8.5 },
  { nome: "Bruno", nota: 5.0 },
  { nome: "Carla", nota: 7.0 },
  { nome: "Davi", nota: 9.5 },
];

const aprovados = alunos
  .filter((aluno) ⇒ aluno.nota ⩾ 6)
  .map((aluno) ⇒ aluno.nome.toUpperCase());

console.log(aprovados.join(", "));

Saída esperada:
ANA, CARLA, DAVI
```

Saídas das suas versões

```
244
ANA, CARLA, DAVI
```

7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

Na minha opinião, a versão funcional é mais fácil de ler porque o código fica mais curto e direto. Porém, acho que a versão com laço é mais fácil de depurar, pois consigo acompanhar cada etapa e verificar os valores durante a execução.

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

Acredito que a versão em Haskell seja mais fácil de entender, porque é mais curta e mostra diretamente o que precisa ser feito. Sobre o tempo de execução, eu aceitaria esse custo em sistemas simples, nos quais o desempenho não é tão importante. Já em sistemas que precisam responder muito rápido, como sistemas bancários em tempo real, eu preferiria uma versão mais eficiente.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi perceber que dá para fazer a mesma coisa com muito menos código usando programação funcional, deixando o programa mais simples e direto.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

O erro de impressão está em b <- h, pois o correto seria b <= h, para selecionar os elementos menores ou iguais ao pivô. A mensagem do compilador indica que ele interpretou h como um valor único, e não como uma lista de valores para percorrer. Corrigindo para b <= h, o código consegue filtrar os elementos corretamente e ordenar a lista.